In [2]:
```
'''Data visualisation means graphical or pictorial
representation of the data using graph, chart,
etc. The purpose of plotting data is to visualise
variation or show relationships between variables.
'''
```

Out[2]:
```
'Data visualisation means graphical or pictorial \nrepresentation of the data
using graph, chart, \netc. The purpose of plotting data is to visualise \nvar
iation or show relationships between variables. \n'
```

In [3]:
```
''' Data presented through visual elements is easy to understand and analyze,
enabling the
Types of data visualization:

    The most common data visualization types are scatter plots, bar charts,
heat maps, line graphs, pie charts, area charts, choropleth maps and
histograms. '''
```

Out[3]:
```
' Data presented through visual elements is easy to understand and analyze,
\nenabling the \nTypes of data visualization:\n        \n    The most common
data visualization types are scatter plots, bar charts, \nheat maps, line gra
phs, pie charts, area charts, choropleth maps and \nhistograms. '
```

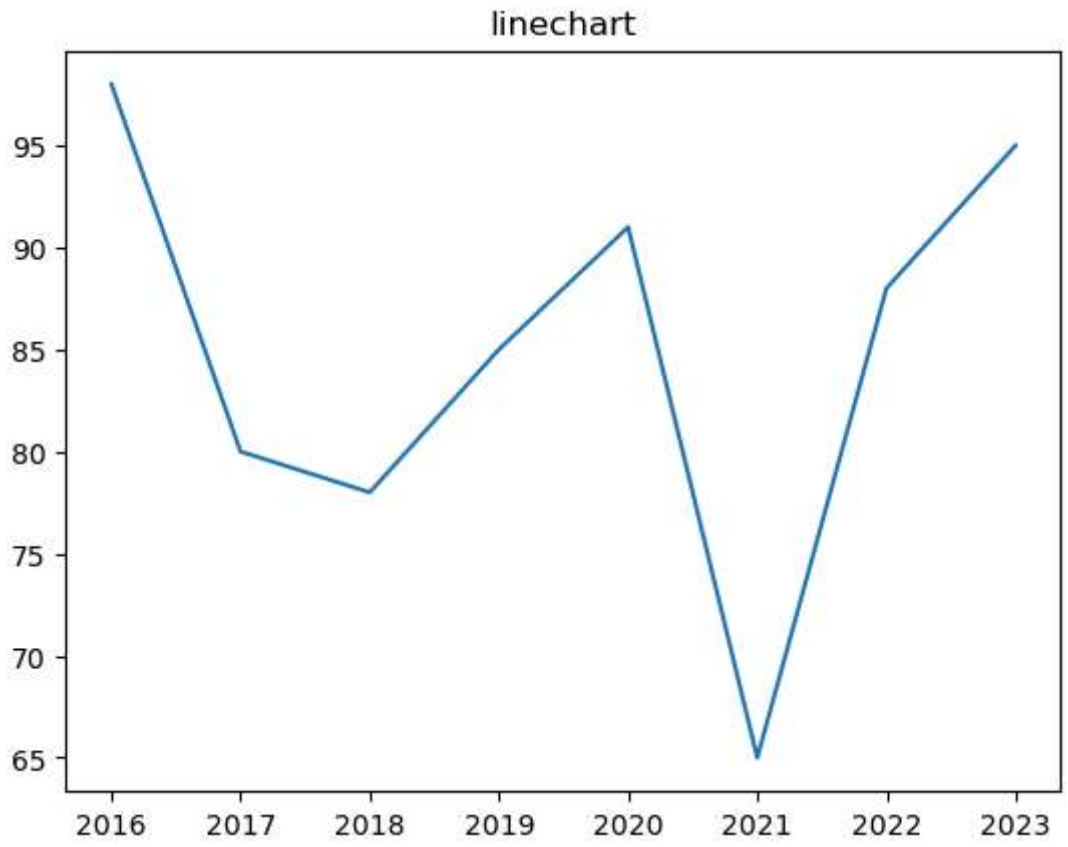
In [4]:
```
import matplotlib.pyplot as plt
```

In [5]:
```
'''Matplotlib library is used for creating static, animated,
and interactive 2D- plots or figures in Python. It can
be installed using the following pip command from the
command prompt:
pip install matplotlib
For plotting using Matplotlib, we need to import its
Pyplot module using the following command:
import matplotlib.pyplot as plt
Here, plt is an alias or an alternative name for
matplotlib.pyplot. We can use any other alias also.
Matplotlib provides a wide range of graphs and charts for data visualization.
Some of the commonly used types of plots supported by Matplotlib include:
1.Line Plot:

plt.plot(x, y)
2. Scatter Plot:

plt.scatter(x, y)
3. Bar Chart:

plt.bar(x, height)
4. Histogram:

plt.hist(data, bins)
5. Pie Chart:

plt.pie(sizes, labels)
6. Box Plot:
plt.boxplot(data)
7.Violin Plot:

plt.violinplot(data)
8.Heatmap:

plt.imshow(data, cmap='viridis', interpolation='nearest')
plt.colorbar()
9.3D Plotting:

from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(x, y, z)
10.Contour Plot:

plt.contour(X, Y, Z)
11.Quiver Plot:

plt.quiver(x, y, u, v)

'''
```

Out[5]:
```
"Matplotlib library is used for creating static, animated, \nand interactive
2D- plots or figures in Python. It can \nbe installed using the following pip
command from the \ncommand prompt:\npip install matplotlib\nFor plotting usin
g Matplotlib, we need to import its \nPyplot module using the following comma
nd:\nimport matplotlib.pyplot as plt\nHere, plt is an alias or an alternative
name for \nmatplotlib.pyplot. We can use any other alias also.\nMatplotlib pr
ovides a wide range of graphs and charts for data visualization. \nSome of th
e commonly used types of plots supported by Matplotlib include:\n1.Line Plo
t:\n\nplt.plot(x, y)\n2. Scatter Plot:\n\nplt.scatter(x, y)\n3. Bar Chart:\n
\nplt.bar(x, height)\n4. Histogram:\n\nplt.hist(data, bins)\n5. Pie Chart:\n
\nplt.pie(sizes, labels)\n6. Box Plot:\nplt.boxplot(data)\n7.Violin Plot:\n\n
plt.violinplot(data)\n8.Heatmap:\n\nplt.imshow(data, cmap='viridis', interpol
ation='nearest')\nplt.colorbar()\n9.3D Plotting:\n\nfrom mpl_toolkits.mplot3d
import Axes3D\nfig = plt.figure()\nax = fig.add_subplot(111, projection='3d')
\nax.scatter(x, y, z)\n10.Contour Plot:\n\nplt.contour(X, Y, Z)\n11.Quiver Pl
ot:\n\nplt.quiver(x, y, u, v)\n\n"
```

In [6]:
```
#plt.plot(x,y) used to create figure
#plt.show() used to display the figure
```

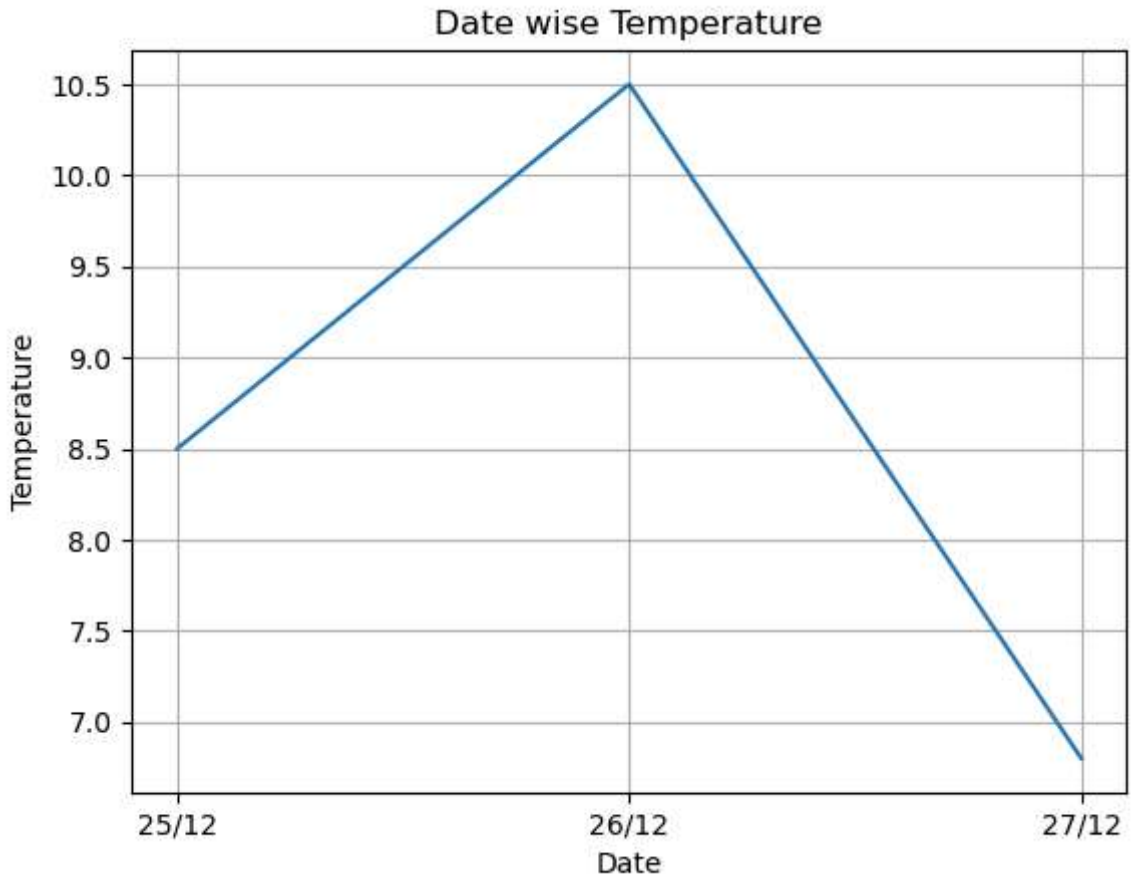
In [7]:
```
year=[2016,2017,2018,2019,2020,2021,2022,2023]
pass_percentage=[98,80,78,85,91,65,88,95]
plt.plot(year,pass_percentage)
plt.title("linechart")
plt.show()
plt.savefig("linechart")
```




```
<Figure size 640x480 with 0 Axes>
```

In [8]:
```
#the plot() function by default plots a line chart.
#A figure can also
#be saved by using savefig() function. The name of the
#figure is passed to the function as parameter.
# plt.savefig('x.png')
```

In [9]:
```
#Adding Labels and Titles:
date=["25/12","26/12","27/12"]
temp=[8.5,10.5,6.8]
plt.plot(date, temp)
plt.xlabel("Date") #add the label on x-axis
plt.ylabel("Temperature") #add the label on y-axis
plt.title("Date wise Temperature") #add the title to the chart
plt.grid(True) #add gridlines to the background
plt.show() #to display the plot
```



In [10]:
```
'''MARKER:A marker is any symbol that represents a data value
in a line chart or a scatter plot.passed as parameter to the plot function
'''
```

Out[10]:
```
'MARKER:A marker is any symbol that represents a data value \nin a line chart
or a scatter plot.passed as parameter to the plot function\n'
```

In [11]:
```
'''Marker Symbol Description Marker Symbol Description
“.” Point “8” octagon
“,” Pixel “s” square
“o” Circle “p” pentagon
“v” triangle_down “P” plus (filled)
“^” triangle_up “*” star
“<” triangle_left “h” hexagon1
“>” triangle_right “H” hexagon2
“1” tri_down “+” plus
“2” tri_up “x” x
“3” tri_left “X” x (filled)
“4” tri_right “D” diamond
```

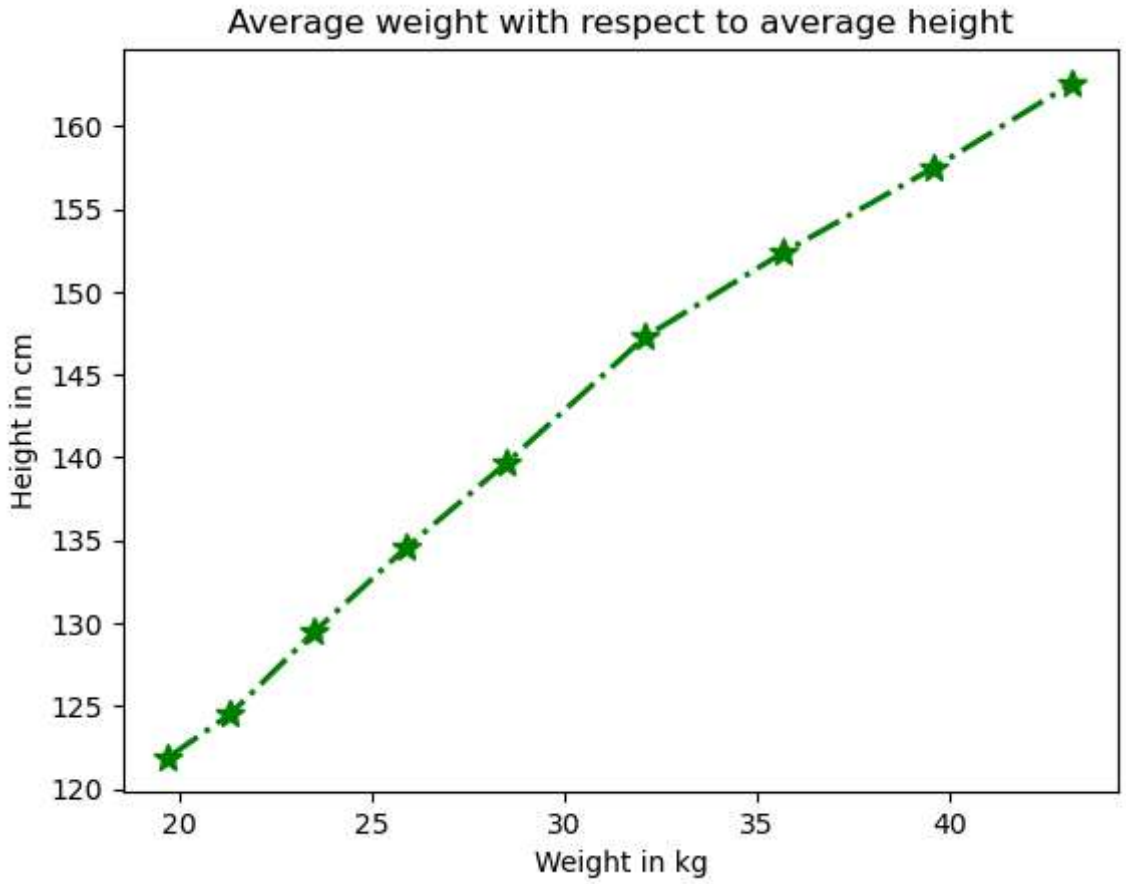
```
  Cell In[11], line 1
    '''Marker Symbol Description Marker Symbol Description
    ^
SyntaxError: incomplete input
```

In [ ]:
```
'''Color:It is also possible to format the plot further by changing
the colour of the plotted data. Table shows the list of
colours that are supported. We can either use character
codes or the color names as values to the parameter
color in the plot().
Table : Colour abbreviations for plotting
Character Colour
‘b’ blue
‘g’ green
‘r’ red
‘c’ cyan
‘m’ magenta
‘y’ yellow
‘k’ black
‘w’ white'''
```

In [ ]:
```
'''Linewidth and Line Style:
The linewidth and linestyle property can be used
to change the width and the style of the line chart.
Linewidth is specified in pixels. The default line width
is 1 pixel showing a thin line. Thus, a number greater
than 1 will output a thicker line depending on the
value provided.
We can also set the line style of a line chart using
the linestyle parameter. It can take a string such as
"solid", "dotted", "dashed" or "dashdot". '''
```

In [ ]:
```
'''Question:Consider the average heights and weights of
persons aged 8 to 16 stored in the following
two lists:
height = [121.9,124.5,129.5,134.6,139.7,147.3,
152.4, 157.5,162.6]
weight= [19.7,21.3,23.5,25.9,28.5,32.1,35.7,39.6,
43.2]
Let us plot a line chart where:
i. x axis will represent weight
ii. y axis will represent height
iii. x axis label should be “Weight in kg”
iv. y axis label should be “Height in cm”
v. colour of the line should be green
vi. use * as marker
vii. Marker size as10
viii. The title of the chart should be “Average
weight with respect to average height”.
ix. Line style should be dashed
x. Linewidth should be 2.'''
```

In [7]:
```
#ans:
import matplotlib.pyplot as plt
import pandas as pd
height=[121.9,124.5,129.5,134.6,139.7,147.3,152.4,157.5,162.6]
weight=[19.7,21.3,23.5,25.9,28.5,32.1,35.7,39.6,43.2]
df=pd.DataFrame({"height":height,"weight":weight})
#Set xlabel for the plot
plt.xlabel('Weight in kg')
#Set ylabel for the plot
plt.ylabel('Height in cm')
#Set chart title:
plt.title('Average weight with respect to average height')
#plot using marker'-*' and line colour as green
plt.plot(df.weight,df.height,marker='*',markersize=10,color='green',linewidth=2
plt.show()
```

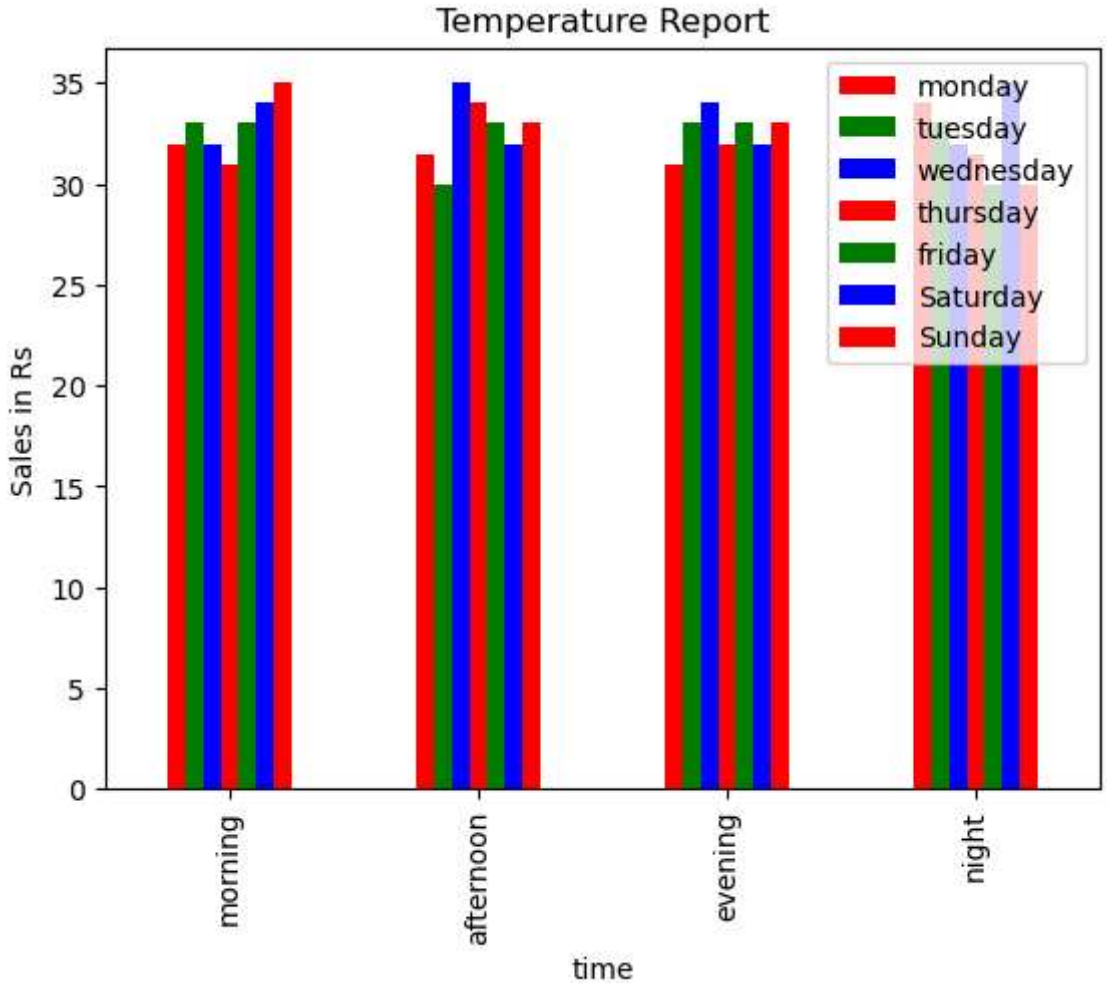


In [18]:
```
'''assignment:
Day-wise mela sales data
Week1 Week2 Week3
5000 4000 4000
5900 3000 5800
6500 5000 3500
3500 5500 2500
4000 3000 3000
5300 4300 5300
7900 5900 6000
Depict the sales for the three weeks using a Line chart. It
should have the following:
i. Chart title as “Mela Sales Report”.
ii. axis label as Days.
iii. axis label as “Sales in Rs”.
Line colours are red for week 1, blue for week 2 and brown
for week 3.'''
```

Out[18]:
```
'assignment:\nDay-wise mela sales data\nWeek1 Week2 Week3\n5000 4000 4000\n59
00 3000 5800\n6500 5000 3500\n3500 5500 2500\n4000 3000 3000\n5300 4300 5300
\n7900 5900 6000\nDepict the sales for the three weeks using a Line chart. I
\nDepict the sales for the three weeks using a Line chart. It \nshould have t
he following:\ni. Chart title as “Mela Sales Report”.\nii. axis label as Day
s.\niii. axis label as “Sales in Rs”.\nLine colours are red for week 1, blue
for week 2 and brown \nfor week 3.'
```

In [ ]:
```
'''The plot() method of Pandas accepts a considerable number of
arguments that can be used to plot a variety of graphs.
It allows customising different plot types by supplying the kind
keyword arguments. The general syntax is: plt.plot(kind),where kind
accepts a string indicating the type of .plot,
as listed in Table 4.5. In addition, we can use the matplotlib.pyplot methods
and functions also along with the plt() method of Pandas objects.
Arguments accepted by kind for different plots
kind = Plot type
line Line plot (default)
bar Vertical bar plot
barh Horizontal bar plot
hist Histogram
box Boxplot
area Area plot
pie Pie plot
scatter Scatter plot '''
'''df.plot(kind="line")
df.plot(kind="bar")
df.plot(kind="hist")
df.plot(kind="box")
df.plot(kind="scatter")'''
```

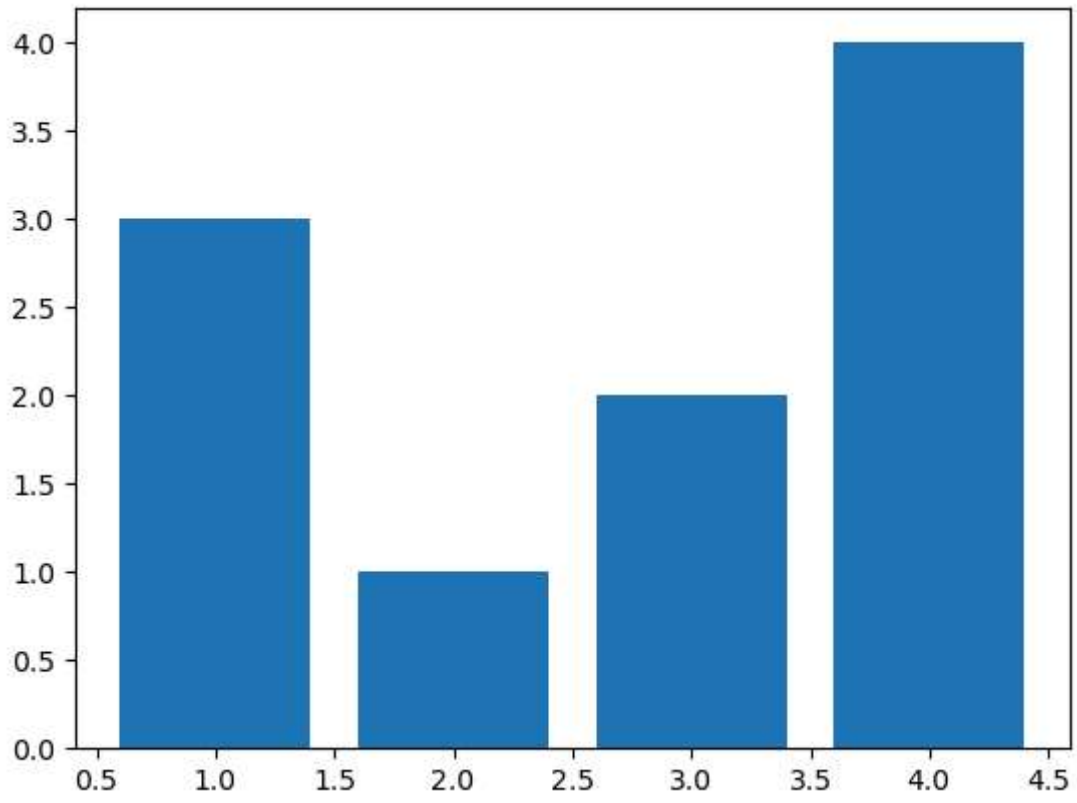
In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
df1=pd.read_csv(r"C:\Users\IIIT\Documents\DSP\Untitled Folder\Temperature.csv")
df1.plot(kind="bar",color=["r","g","b"],x="time")
plt.title('Temperature Report')
plt.xlabel('time')
plt.ylabel('Sales in Rs')
plt.show()
```



In [ ]:
```
```

In [25]:
```
'''#bar Chart:In order to show comparisons, we prefer Bar charts.
#assignment bar chart:write the Python script to
display Bar plot for the “MelaSales.csv” file with column Day on x axis

Day-wise sales data along with Day’s names
Week1 Week2 Week3 Day
5000 4000 4000 Monday
5900 3000 5800 Tuesday
6500 5000 3500 Wednesday
3500 5500 2500 Thursday
4000 3000 3000 Friday
5300 4300 5300 Saturday
7900 5900 6000 Sunday '''
```

In [58]:
```
#another way of creating bar chart
import numpy as np
x=np.array([1,2,3,4])
y=np.array([3,1,2,4])
plt.bar(x,y)
plt.show()
```

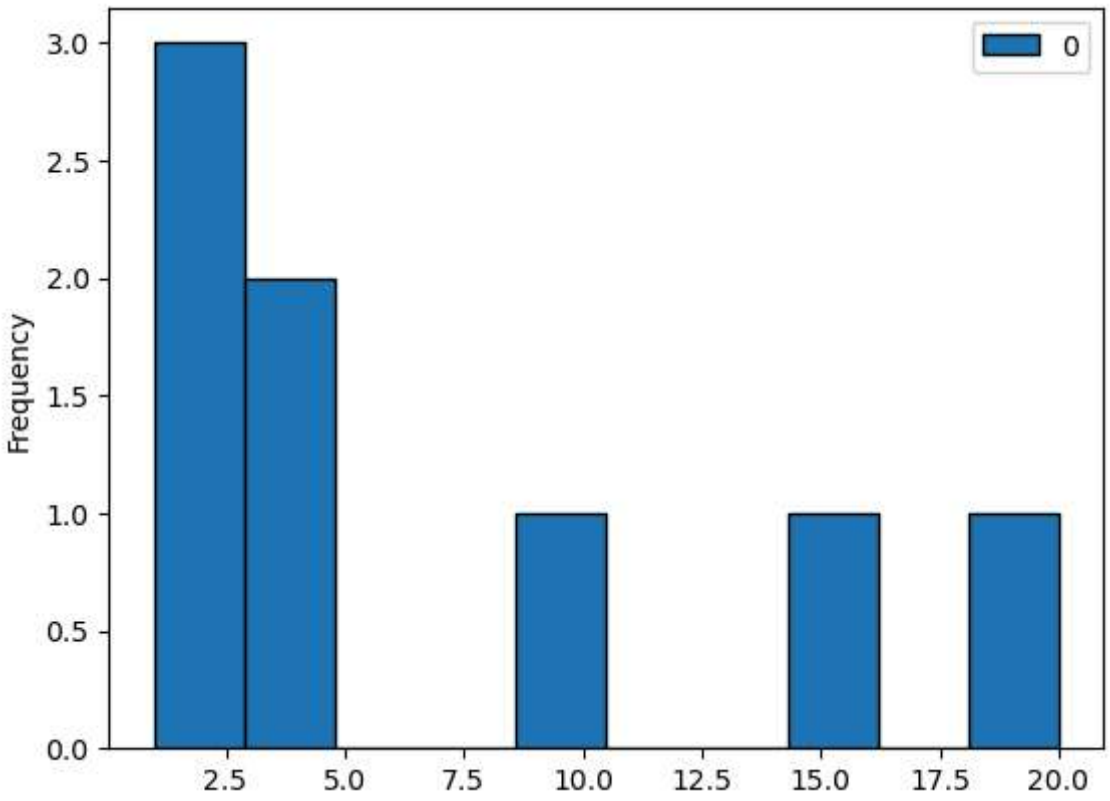


In [17]:
```
#histogram is used to divide the data into set of bins..it will count the_
#numbers which is existing in between a range
'''A histogram is a graphical representation of the distribution of numerical
data.
It consists of a series of bars, where the height of each bar corresponds
to the
frequency or
count of data points falling within a specific interval or bin.'''
```

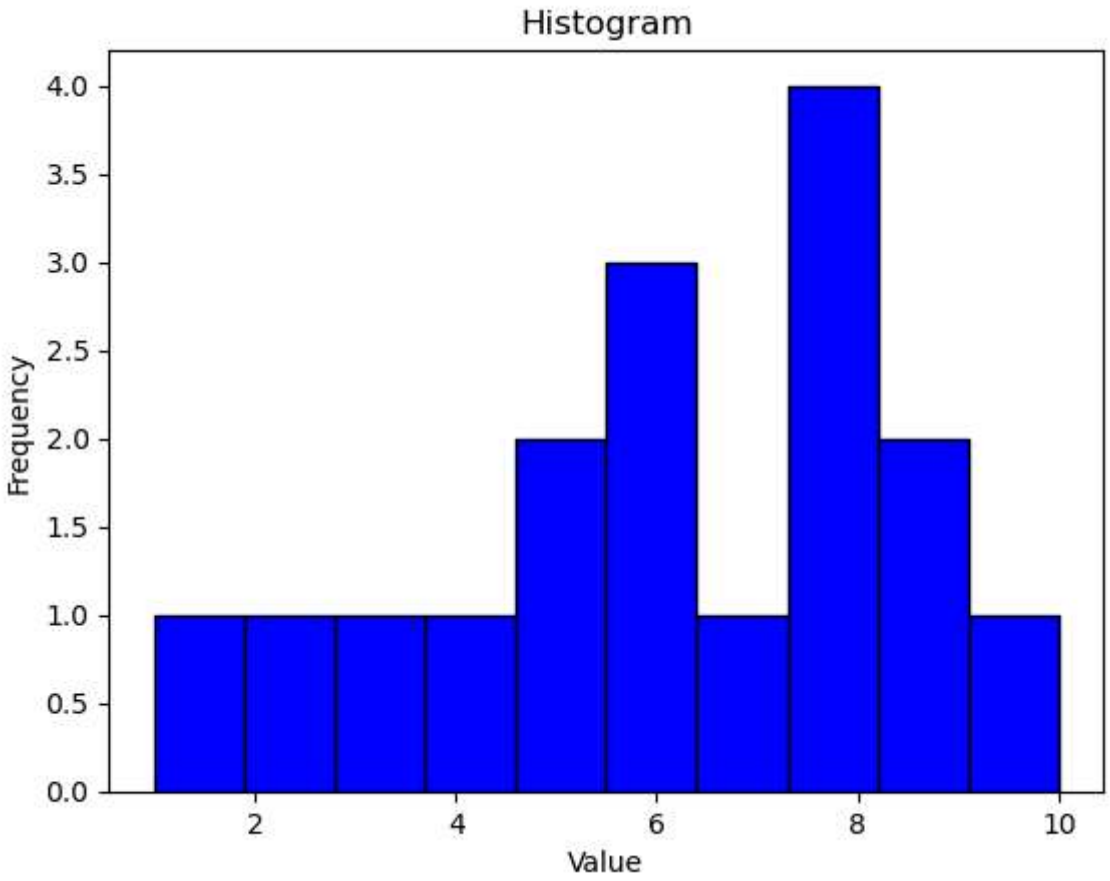
Out[17]:
```
'A histogram is a graphical representation of the distribution of numerical d
ata. \nIt consists of a series of bars, where the height of each bar correspo
nds to the frequency or\ncount of data points falling within a specific inter
val or bin.'
```

In [16]:
```
#another of creating histogram
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
x=np.array([1,2,3,4,10,1,15,20])
df2=pd.DataFrame(x)
df2.plot(kind="hist",edgecolor="black")
```

Out[16]:
```
<Axes: ylabel='Frequency'>
```



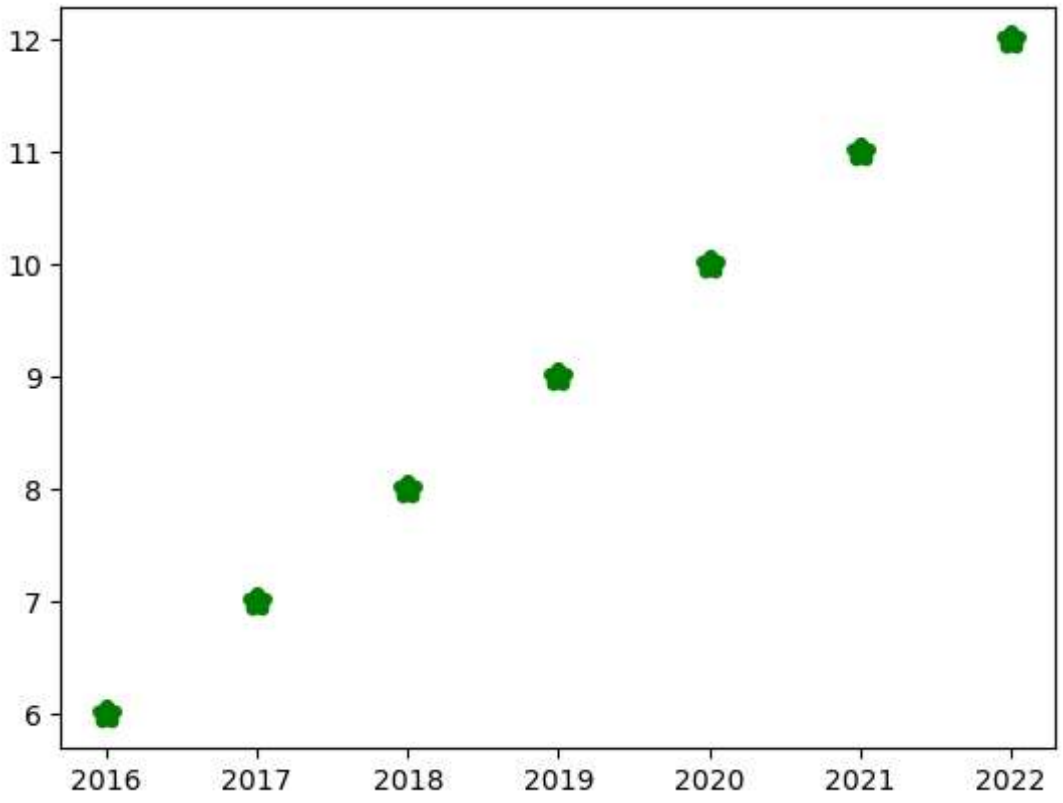
In [12]:
```
import matplotlib.pyplot as plt

# Sample data
data = [1, 2, 3, 4, 5, 5, 6, 6, 6, 7, 8, 8, 8, 8, 9, 9, 10]

# Create histogram
plt.hist(data, bins=10, color='blue', edgecolor='black')

# Add labels and title
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram')

# Display the plot
plt.show()
```



In [ ]:
```
 #Plotting Scatter Chart
'''A scatter chart is a two-dimensional data visualisation
method that uses dots to represent the values obtained
for two different variables —one plotted along the x-axis
and the other plotted along the y-axis.
Scatter plots are used when you want to show the
relationship between two variables. Scatter plots are
sometimes called correlation plots because they show
how two variables are correlated. Additionally, the size,
shape or color of the dot could represent a third (or even
fourth variable).'''
```

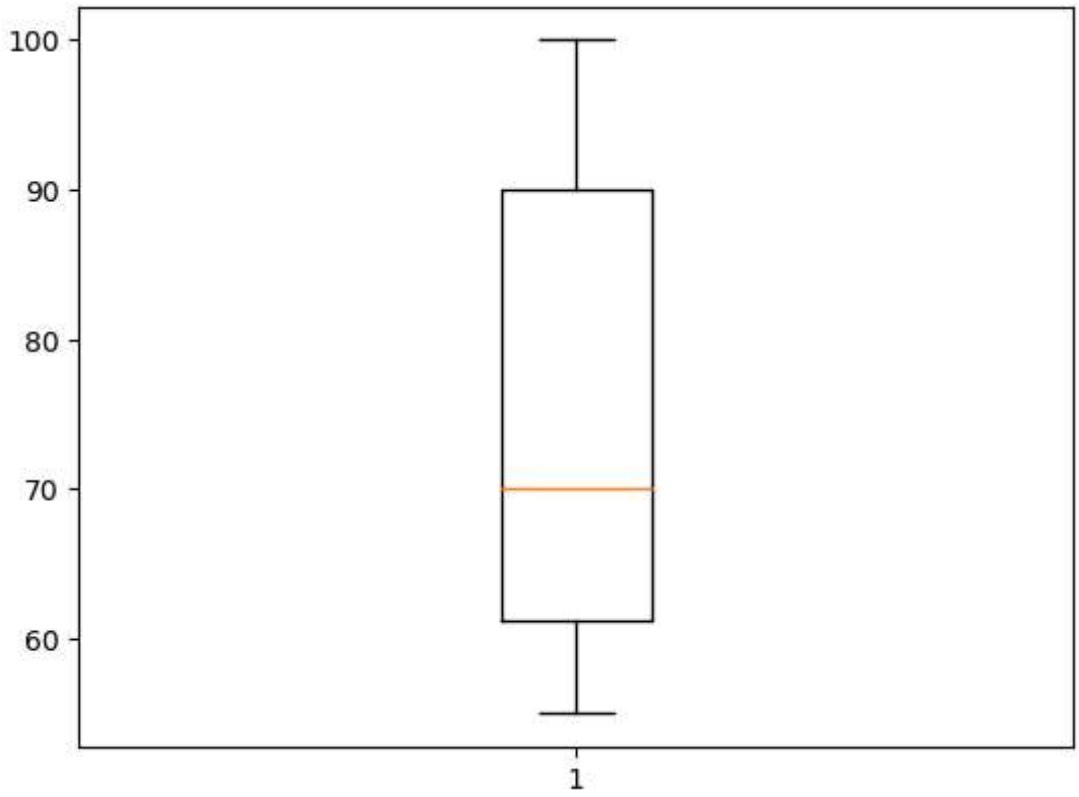
In [80]:
```
year=[2016,2017,2018,2019,2020,2021,2022]
population_in_cr=[6,7,8,9,10,11,12]
plt.scatter(year,population_in_cr,color="green",marker="*",linewidth=5)
```

Out[80]:
```
<matplotlib.collections.PathCollection at 0x2141aa10a10>
```



In [ ]:
```
#Assignment Draw a scatter plot
#to show a relationship between the discount
#offered and sales made.
```

In [81]:
```
#box plot
'''A Box Plot is the visual representation of the
statistical summary of a given data set. The summary
includes Minimum value, Quartile 1, Quartile 2, Median,
Quartile 4 and Maximum value. Quartiles are the measures which divide the data
into four equal parts, and each part contains an equal
number of observations. Calculating quartiles requires
calculation of median.'''
marks=[65,55,100,95,75,60]
plt.boxplot(marks)
plt.show()
```



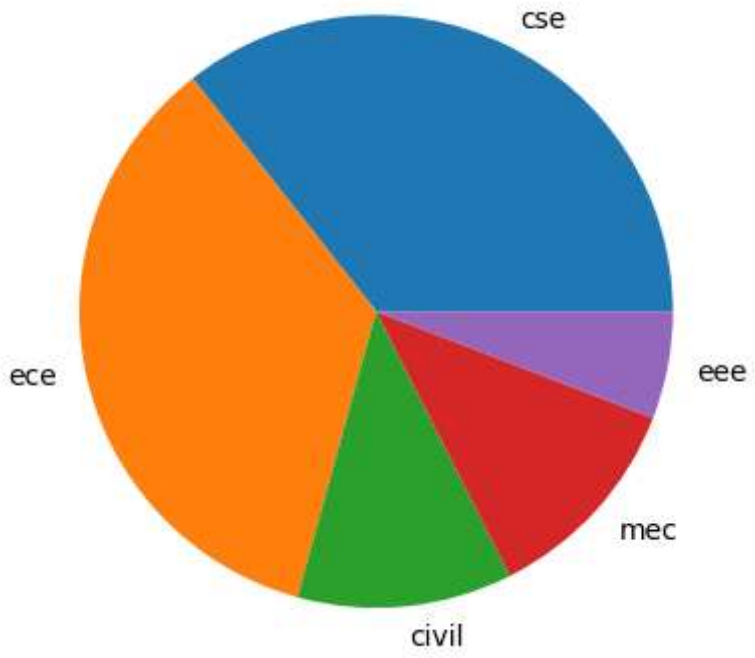
In [ ]:
```
#ANOTHER WAY OF PLOTTING BOX PLOT IS df.plot(kind="box")
```

In [ ]:
```
'''ASSIGNMENT: In order to assess the performance of students
of a class in the annual examination, the
class teacher stored marks of the students in
all the 5 subjects in a CSV “Marks.csv” file
as shown in Table. Plot the data using
boxplot and perform a comparative analysis
of performance in each subject
TABLE:
Marks obtained by students in five subjects
Name  English Maths Hindi Science Social_Studies
Rishika Batra 95 95 90 94 95
Waseem Ali 95 76 79 77 89
Kulpreet Singh 78 81 75 76 88
Annie Mathews 88 63 67 77 80
Shiksha 95 55 51 59 80
Naveen Gupta 82 55 63 56 74
Taleem Ahmed 73 49 54 60 77
Pragati Nigam 80 50 51 54 76
Usman Abbas 92 43 51 48 69
Gurpreet Kaur 60 43 55 52 71
Sameer Murthy 60 43 55 52 71
Angelina 78 33 39 48 68
Angad Bedi 62 43 51 48 54'''
```

In [ ]:
```
#SOLUTION
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
data= pd.read_csv('Marks.csv')
df= pd.DataFrame(data)
df.plot(kind='box')
#set title,xlabel,ylabel
plt.title('Performance Analysis')
plt.xlabel('Subjects')
plt.ylabel('Marks')
plt.show()
```

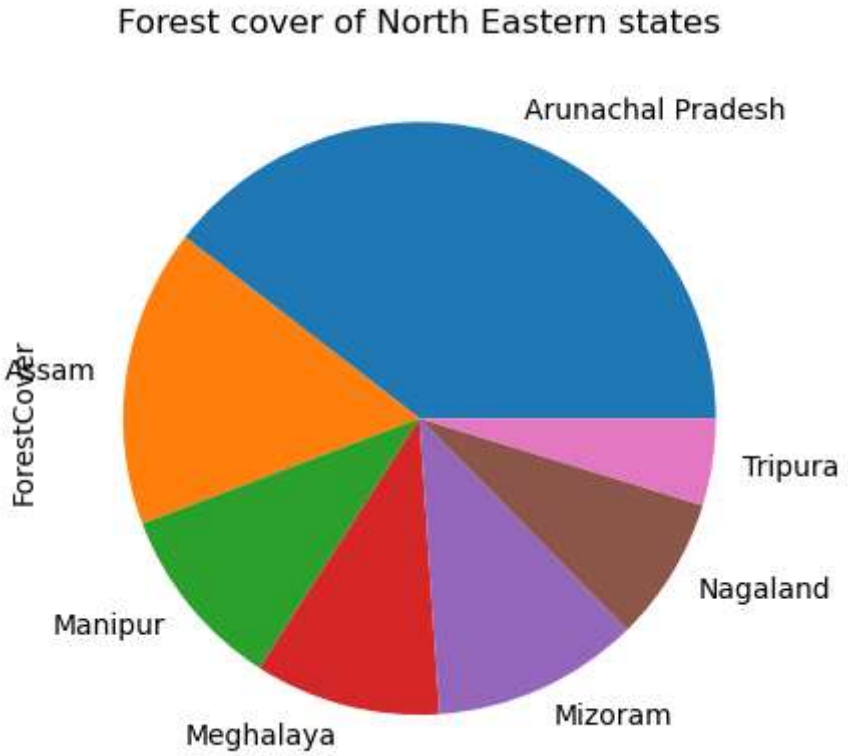
In [ ]:
```
# Pie:  is a type of graph in which a circle is divided into
'''different sectors and each sector represents a part of
the whole. A pie plot is used to represent numerical
data proportionally. To plot a pie chart, either column
label y or 'subplots=True' should be set while using
df.plot(kind='pie') . If no column reference is passed and
subplots=True, a 'pie' plot is drawn for each numerical
column independently.'''
```

In [82]:
```
x=np.array([365,360,120,120,60])
plt.pie(x,labels=["cse","ece","civil","mec","eee"])
plt.show()
```



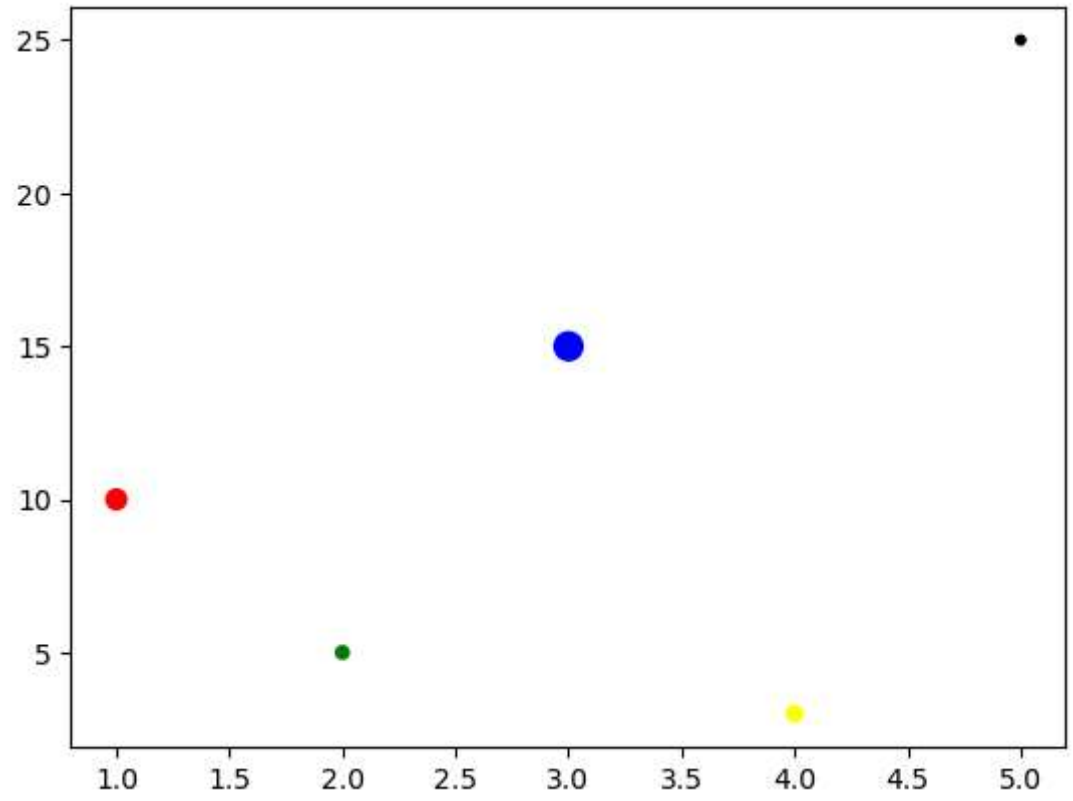
In [ ]:
```
'''Assignment: Let us consider the dataset of Table
showing the forest cover of north eastern
states that contains geographical area and
corresponding forest cover in sq km along
with the names of the corresponding states.

Table: Forest cover of north eastern states
State               GeoArea ForestCover
Arunachal Pradesh   83743   67353
Assam               78438   27692
Manipur             22327   17280
Meghalaya           22429   17321
Mizoram             21081   19240
Nagaland            16579   13464
Tripura             10486   8073
```

In [85]:
```
#sol
import pandas as pd
import matplotlib.pyplot as plt
df=pd.DataFrame({'GeoArea':[83743,78438,22327,22429,21081,16579,10486],'ForestC
index=['Arunachal Pradesh','Assam','Manipur','Meghalaya',
'Mizoram','Nagaland','Tripura'])
df.plot(kind='pie',y='ForestCover',title='Forest cover of North Eastern states'
plt.show()
```



In [ ]:
```
'''BUBBLE PLOTS:
Bubble charts are a type of scatter plot where data points are
represented as bubbles, and the size and sometimes the color of
the bubbles convey additional information. They are particularly
useful for visualizing three-dimensional data sets where each data
point has three numeric values: the x-coordinate, the y-coordinate,
and a third value that determines the size or color of the bubble.
They allow viewers to quickly identify clusters of data points, outliers,
or trends based on the size and/or color of the bubbles.
Comparisons between groups or categories can also be made visually by
observing differences in bubble sizes and colors.
explore relationships between multiple variables in a dataset.'''
```

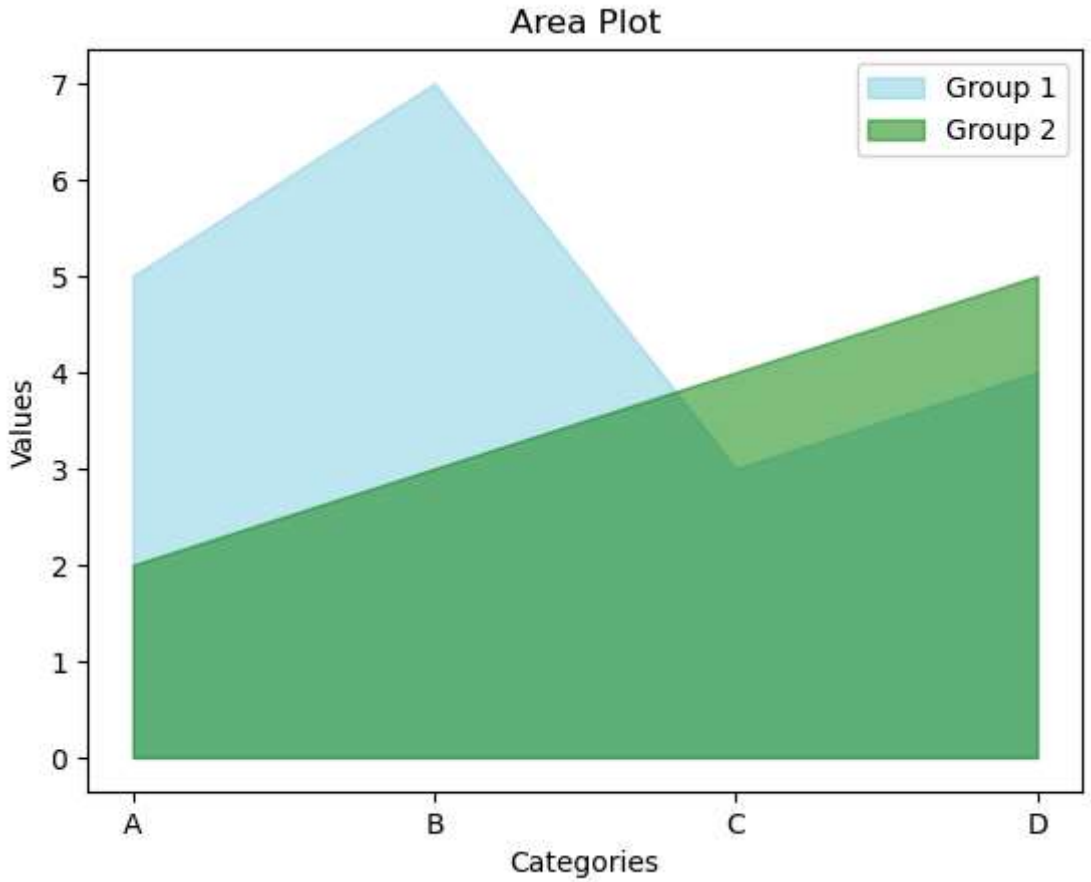
In [13]:
```
import pandas as pd
import matplotlib.pyplot as plt
x=[1,2,3,4,5]
y=[10,5,15,3,25]
plt.scatter(x,y,sizes=[50,20,100,30,10],color=["red","green","blue","yellow","b
plt.show()
```



In [ ]:
```
'''An area plot, also known as a stacked line plot or a filled line plot,
is a type of plot where the area under the line is filled with color.
This plot is useful for showing how
different parts of a whole change over time or across categories.
An area chart is a type of data visualization that displays
quantitative data over time or categories. It is similar to a line chart,
but the area between the line and the x-axis is filled with color,
visually representing the data's magnitude.
Area charts are commonly used to show multiple variables'
cumulative totals or compare
the proportions of different categories.'''
```

In [ ]:
```
#alpha=0.4 is used in the fill_between() function to set the transparency of th
#A value of 0.0 means completely transparent, while 1.0 means completely opaque
```

In [22]:
```
import matplotlib.pyplot as plt

# Sample data
categories = ['A', 'B', 'C', 'D']
values1 = [5, 7, 3, 4]
values2 = [2, 3, 4, 5]

# Create area plot
plt.fill_between(categories, values1, color="skyblue",alpha=0.5,label='Group 1'
plt.fill_between(categories, values2, color="green", alpha=0.5,label='Group 2')

# Add labels and title
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Area Plot')

# Add legend
plt.legend()

# Display the plot
plt.show()
```

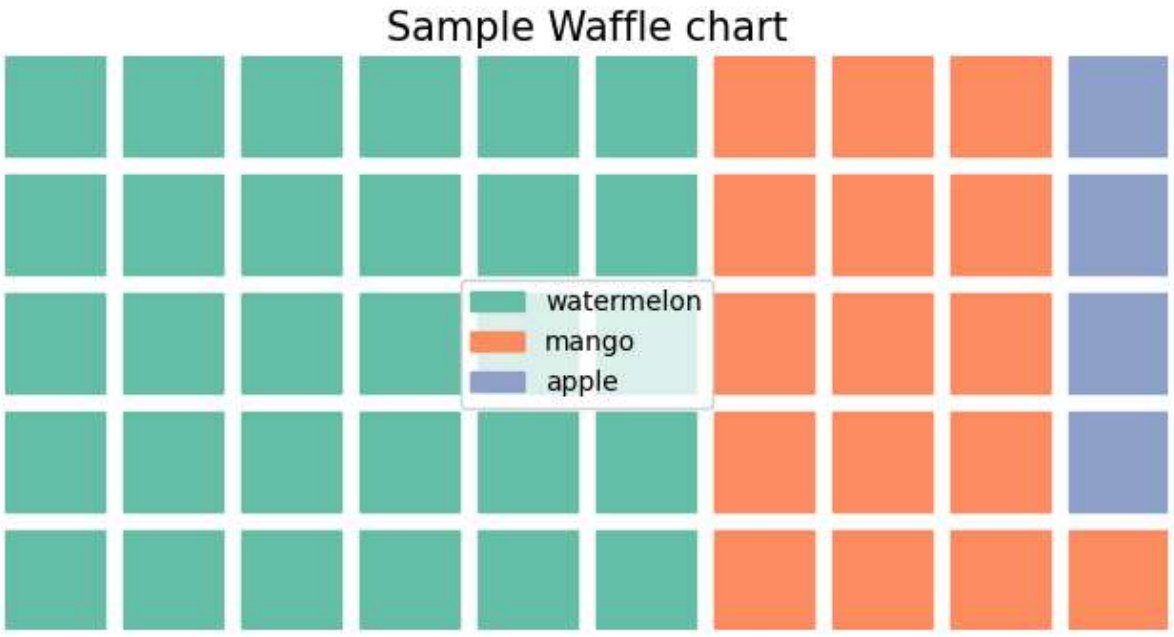


In [ ]:
```
'''waffle chart: The waffle chart is also known by the name Grid Plot.
A waffle chart is represented by a square or rectangular grid of cells,
each corresponding to 1 unit or 1%. The cells are represented with colors
to differentiate between categories.
A waffle chart can include multiple categories.'''
```

In [28]:
```
pip install pywaffle
```

```
Note: you may need to restart the kernel to use updated packages.Collecting p
ywaffle
```

In [11]:
```
import matplotlib.pyplot as plt
from pywaffle import Waffle

fig = plt.figure(FigureClass=Waffle,rows=5,columns=10, values=[30, 16, 4],title
```



In [ ]:
```
''' visualisation method that displays how frequently words appear
in a given body of text,
by making the size of each word proportional to its frequency.
All the words are then arranged in a cluster or cloud of words.
Alternatively, the words can also be
arranged in any format:
horizontal lines, columns or within a shape.

Word Clouds can also be used to display words that have meta-data
assigned to them. For example, in a Word Cloud of all the World's countries,
the population could be assigned to each country's name to determine its size.

Colour used on Word Clouds is usually meaningless and is primarily aesthetic,
but it can be used to categorise words or to display another data variable.'''
```

In [35]:
```
pip install wordcloud
```

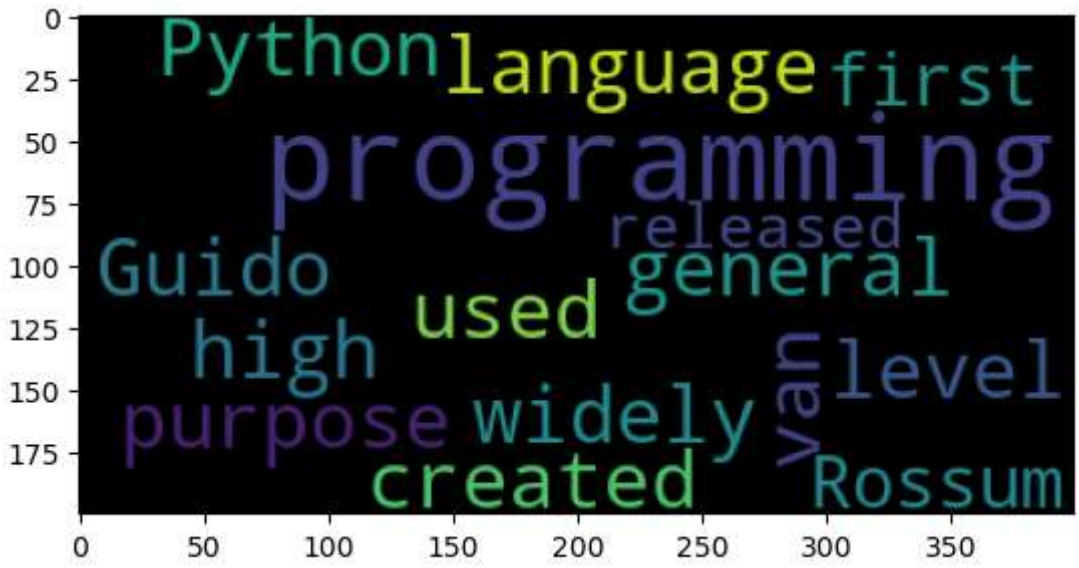
```
Collecting wordcloudNote: you may need to restart the kernel to use updated
packages.

  Obtaining dependency information for wordcloud from https://files.pythonh
osted.org/packages/f5/b0/247159f61c5d5d6647171bef84430b7efad4db504f02296740
24f3a4f7f2/wordcloud-1.9.3-cp311-cp311-win_amd64.whl.metadata (https://file
s.pythonhosted.org/packages/f5/b0/247159f61c5d5d6647171bef84430b7efad4db504
f0229674024f3a4f7f2/wordcloud-1.9.3-cp311-cp311-win_amd64.whl.metadata)
  Downloading wordcloud-1.9.3-cp311-cp311-win_amd64.whl.metadata (3.5 kB)
Requirement already satisfied: numpy>=1.6.1 in c:\users\iiit\anaconda3\lib
\site-packages (from wordcloud) (1.24.3)
Requirement already satisfied: pillow in c:\users\iiit\anaconda3\lib\site-p
ackages (from wordcloud) (10.2.0)
Requirement already satisfied: matplotlib in c:\users\iiit\anaconda3\lib\si
te-packages (from wordcloud) (3.7.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\iiit\anaconda3
\lib\site-packages (from matplotlib->wordcloud) (1.0.5)
Requirement already satisfied: cycler>=0.10 in c:\users\iiit\anaconda3\lib
\site-packages (from matplotlib->wordcloud) (0.11.0)
```

In [4]:
```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
```

In [ ]:
```
```

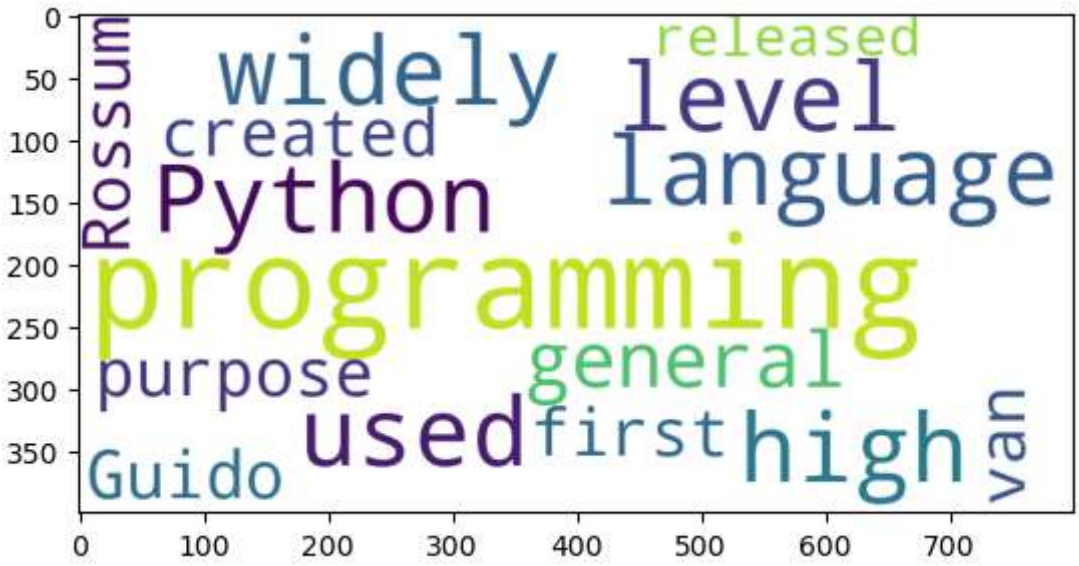
In [9]:
```
text = "Python is a widely used high-level programming language for general-pur
a=WordCloud().generate(text)
plt.imshow(a)
plt.show()
```

Out[9]:
```
<matplotlib.image.AxesImage at 0x2418afb2c50>
```



In [ ]:
```
'''We import the WordCloud class from the wordcloud library and
matplotlib.pyplot for visualization.
We define a sample text (text) containing words and their frequencies.
We create a WordCloud object with specified width, height,
and background color, and generate the word cloud from the text using
the generate() method.
We use matplotlib.pyplot.imshow() to display the word cloud
Finally, we show the word cloud using plt.show().
You can customize various parameters such as the font size, maximum
words, color scheme, and more to create word clouds tailored to
your specific needs. Additionally, you can read text data from
files or process real-world data to generate word clouds from larger datasets.'
```

In [40]:
```
text = "Python is a widely used high-level programming language for general-pur
a=WordCloud(width=800, height=400, background_color='white').generate(text)
plt.imshow(a)
plt.show()
```

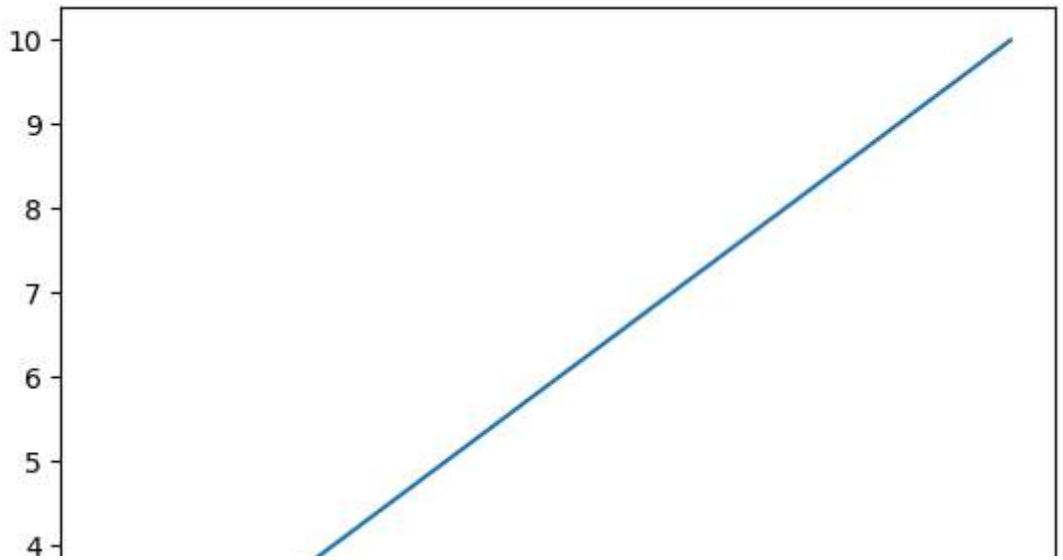


In [ ]:
```
'''seaborn: Matplotlib and Seaborn are both Python libraries used for data visu
but they serve slightly different purposes and have different strengths.
Here's a comparison of the two:

Matplotlib: Matplotlib is a low-level library that provides a wide variety of p
functions for creating basic plots such as line plots, scatter plots, bar plots
histograms, etc. It offers fine-grained control over plot elements and is highl
Seaborn: Seaborn is a higher-level library that builds on top of Matplotlib.
It provides a more concise and intuitive interface for creating complex statist
Seaborn automates many aspects of plot creation, making it easier to generate
informative plots with less code.'''
```

In [ ]:
```
'''Seaborn provides a wide range of plotting functions for creating various typ

Histograms and Kernel Density Estimation (KDE) Plots:

sns.histplot(): Creates a histogram with optional KDE curve overlaid.
sns.kdeplot(): Plots the kernel density estimate of a univariate distribution.
Scatter Plots:

sns.scatterplot(): Creates a scatter plot to visualize the relationship betwee
Line Plots:

sns.lineplot(): Draws a line plot to represent the relationship between two var
Bar Plots:

sns.barplot(): Creates a bar plot to visualize the distribution of a categorica
Count Plots:

sns.countplot(): Draws a bar plot to show the counts of observations in each ca
Box Plots and Violin Plots:

sns.boxplot(): Draws a box plot to represent the distribution of a categorical
sns.violinplot(): Creates a violin plot to visualize the distribution of a nume
Pair Plots:

sns.pairplot(): Draws pairwise scatter plots for multiple pairs of variables an
Joint Plots:

sns.jointplot(): Creates a joint plot to visualize the joint distribution of tw
Heatmaps:

sns.heatmap(): Draws a heatmap to represent the correlation matrix or other 2D
Facet Grids:

sns.FacetGrid(): Facilitates the creation of multiple plots (e.g., scatter plot
These are just a few examples of the plots available in Seaborn. Each plotting


'''
```

In [25]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
x_values = [1, 2, 3, 4, 5]
y_values = [2, 4, 6, 8, 10]

# Create line plot using Seaborn
sns.lineplot(x=x_values, y=y_values)

# Add labels and title
'''plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Line Plot')

# Display the plot
plt.show()'''
```

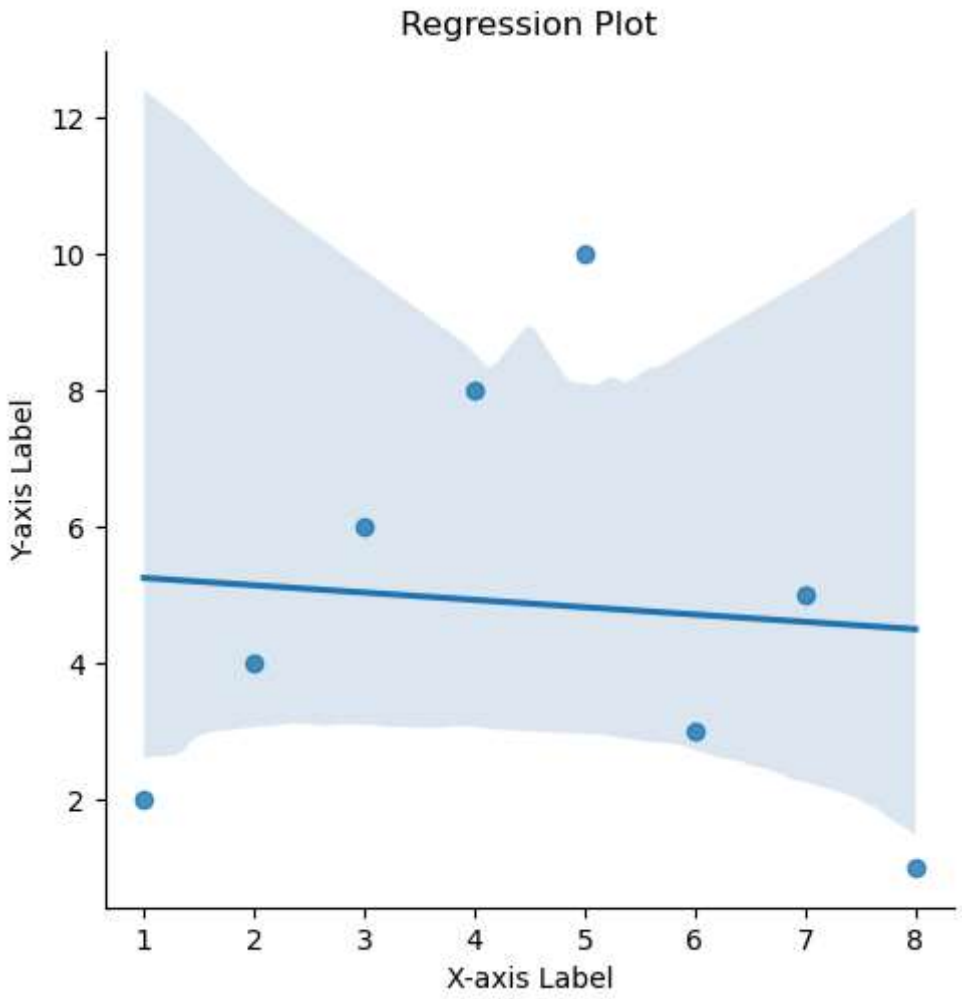
Out[25]:
```
"plt.xlabel('X-axis Label')\nplt.ylabel('Y-axis Label')\nplt.title('Line Pl
ot')\n\n# Display the plot\nplt.show()"
```



In [ ]:
```
'''Regression plots as the name suggests creates a regression line between
2 parameters and helps to visualize their linear relationships.
Seaborn is not only a visualization library but also a provider of built-in dat
Here, we will be working with one of such datasets in seaborn named ‘tips’.
The tips dataset contains information about the people who probably had food at
the restaurant and whether or not they left a tip. It also provides information
the gender of the people, whether they smoke, day, time and so on.
Regression plots in seaborn can be easily implemented with the help of the lmpl
lmplot() can be understood as a function that basically creates a linear model
lmplot() makes a very simple linear regression plot.It creates a scatter plot w
linear fit on top of it.

A regression plot is a type of data visualization that is used to visually
assess the relationship between two variables and to model and understand the
relationship between them. In particular, it is used to examine whether there i
a linear relationship between the variables and to fit a regression model to th
 regression model is a statistical method used to examine the relationship betw
 or more independent variables (predictors) and a dependent variable (outcome).
 lmplot(): This function creates a scatter plot with a linear regression line f
'''
```

In [29]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
x_values = [1, 2, 3, 4, 5,6,7,8]
y_values = [2, 4, 6, 8, 10,3,5,1]

# Create regression plot using lmplot
sns.lmplot(x='x', y='y', data=pd.DataFrame({'x': x_values, 'y': y_values}))

# Add labels and title
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Regression Plot')

# Display the plot
plt.show()
```

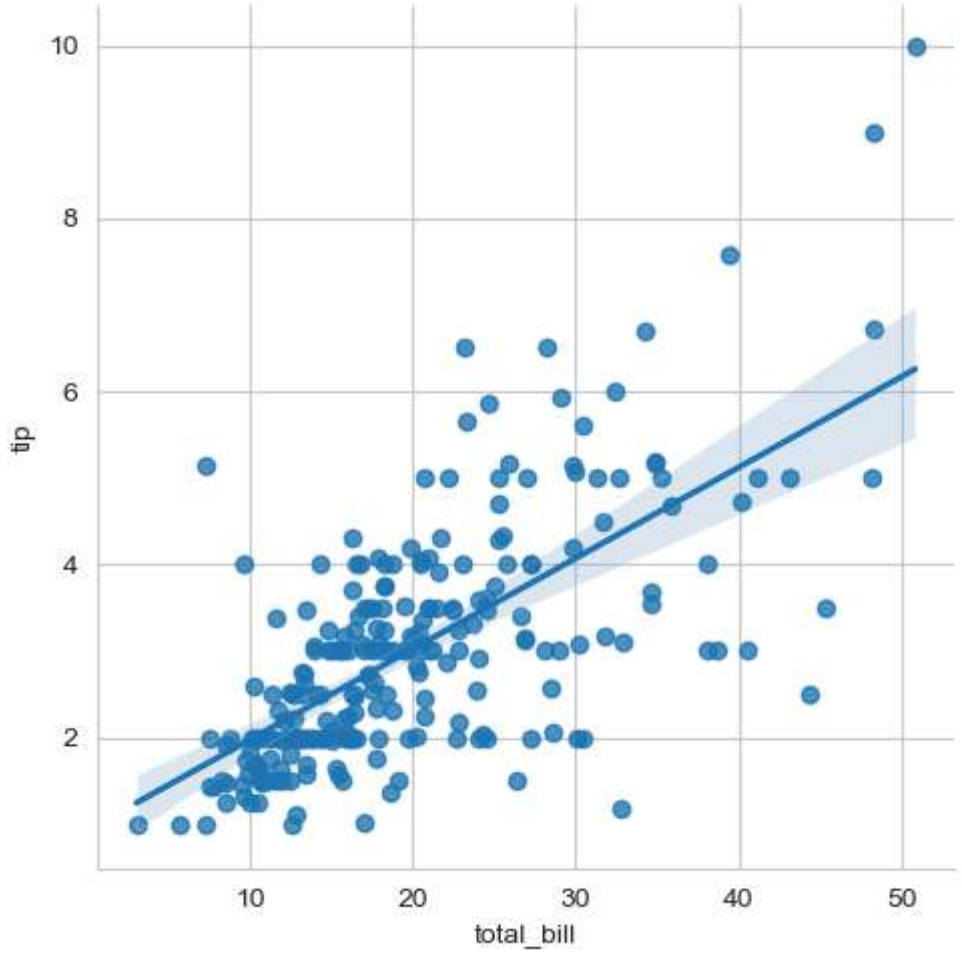
```
C:\Users\IIIT\anaconda3\Lib\site-packages\seaborn\axisgrid.py:118: UserWarnin
g: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```



In [31]:
```
import seaborn as sns

# load the dataset
dataset = sns.load_dataset('tips')

# the first five entries of the dataset
dataset.head()
sns.set_style('whitegrid')
sns.lmplot(x ='total_bill', y ='tip', data = dataset)
```

```
C:\Users\IIIT\anaconda3\Lib\site-packages\seaborn\axisgrid.py:118: UserWarnin
g: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```

Out[31]: <seaborn.axisgrid.FacetGrid at 0x2418eaaff50>



In [ ]:
```
'''folium: Folium is a Python library used for visualizing geospatial data.
It builds on the capabilities of the Leaflet.js library and provides a simple
way to create interactive maps directly within Python. Folium allows users to
create maps with various tilesets, markers, polygons, popups, and other
customizable features.'''
```

In [3]:

```
pip install folium
```

```
Collecting folium
  Obtaining dependency information for folium from https://files.pythonhoste
d.org/packages/b9/98/9ba4b9d2d07dd32765ddb4e4c189dcbdd7dca4d5a735e2e4ea756f40
c36b/folium-0.16.0-py2.py3-none-any.whl.metadata (https://files.pythonhosted.
org/packages/b9/98/9ba4b9d2d07dd32765ddb4e4c189dcbdd7dca4d5a735e2e4ea756f40c3
6b/folium-0.16.0-py2.py3-none-any.whl.metadata)
  Downloading folium-0.16.0-py2.py3-none-any.whl.metadata (3.6 kB)
Collecting branca>=0.6.0 (from folium)
  Obtaining dependency information for branca>=0.6.0 from https://files.pytho
nhosted.org/packages/17/ce/14166d0e273d12065516625fb02426350298e7b4ba59198b5f
e454b46202/branca-0.7.1-py3-none-any.whl.metadata (https://files.pythonhoste
d.org/packages/17/ce/14166d0e273d12065516625fb02426350298e7b4ba59198b5fe454b4
6202/branca-0.7.1-py3-none-any.whl.metadata)
  Downloading branca-0.7.1-py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: jinja2>=2.9 in c:\users\iiit\anaconda3\lib\sit
e-packages (from folium) (3.1.2)
Requirement already satisfied: numpy in c:\users\iiit\anaconda3\lib\site-pack
ages (from folium) (1.24.3)
Requirement already satisfied: requests in c:\users\iiit\anaconda3\lib\site-p
ackages (from folium) (2.31.0)
Requirement already satisfied: xyzservices in c:\users\iiit\anaconda3\lib\sit
e-packages (from folium) (2022.9.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\iiit\anaconda3\lib
\site-packages (from jinja2>=2.9->folium) (2.1.1)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\iiit\anac
onda3\lib\site-packages (from requests->folium) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\iiit\anaconda3\lib\si
te-packages (from requests->folium) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\iiit\anaconda3
\lib\site-packages (from requests->folium) (1.26.16)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\iiit\anaconda3
\lib\site-packages (from requests->folium) (2024.2.2)
Downloading folium-0.16.0-py2.py3-none-any.whl (100 kB)
   ---------------------------------------- 0.0/100.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/100.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/100.0 kB ? eta -:--:--
   ---- ----------------------------------- 10.2/100.0 kB ? eta -:--:--
   ---- ----------------------------------- 10.2/100.0 kB ? eta -:--:--
   ---- ----------------------------------- 10.2/100.0 kB ? eta -:--:--
   ----------- -------------------------- 30.7/100.0 kB 119.1 kB/s eta 0:00:
01
   ----------- -------------------------- 30.7/100.0 kB 119.1 kB/s eta 0:00:
01
   ---------------------- --------------- 61.4/100.0 kB 172.4 kB/s eta 0:00:
01
   ---------------------- --------------- 61.4/100.0 kB 172.4 kB/s eta 0:00:
01
   ------------------------------ ------- 81.9/100.0 kB 183.3 kB/s eta 0:00:
01
   ------------------------------ ------- 81.9/100.0 kB 183.3 kB/s eta 0:00:
01
   ---------------------------------- --- 92.2/100.0 kB 174.7 kB/s eta 0:00:
01
   ---------------------------------- --- 92.2/100.0 kB 174.7 kB/s eta 0:00:
01
   ---------------------------------- --- 92.2/100.0 kB 174.7 kB/s eta 0:00:
01
```

```
   ---------------------------------- --- 92.2/100.0 kB 174.7 kB/s eta 0:00:
01
   ---------------------------------------- 100.0/100.0 kB 136.8 kB/s eta 0:00:
00
Downloading branca-0.7.1-py3-none-any.whl (25 kB)
Installing collected packages: branca, folium
Successfully installed branca-0.7.1 folium-0.16.0
Note: you may need to restart the kernel to use updated packages.
```

In [11]:
```
#creating empty world map
import folium

m = folium.Map()
m
```

Out[11]: Make this Notebook Trusted to load map: File -> Trust Notebook

In [8]:
```
'''Add a Geolocation and Adjust the Zoom Level:
You may not always have data that concerns the whole world. If you want to
display only a specific area of the globe,
then you can add values to another parameter when creating the Map object:'''
import folium

# Create a map centered at a specific location
m = folium.Map(location=[51.5074, -0.1278], zoom_start=10)

# Add a marker at a specific location
folium.Marker(location=[51.5074, -0.1278], popup='London').add_to(m)

# Display the map
m
```

Out[8]: Make this Notebook Trusted to load map: File -> Trust Notebook

In [ ]:
```
'''1. **Importing Folium**: We begin by importing the Folium library,
which we'll use to create and manipulate the map.

2. **Creating the Map**: We create a new map object `m` using the
`folium.Map()` function. We specify the `location` parameter to set
the center of the map, which is done by providing latitude and
longitude coordinates. Here, we've set it to London, England, with
coordinates `[51.5074, -0.1278]`. We also specify the `zoom_start` parameter,
which determines the initial zoom level of the map. In this case, we've set
it to `10`.

3. **Adding a Marker**: We add a marker to the map using the `folium.Marker()`
function. We specify the `location` parameter to set the position of the marker
which is again specified using latitude and longitude coordinates.
We also provide a `popup` parameter to add a popup message to the marker,
which will be displayed when the marker is clicked. Here, we've set the popup
message to ``'London'``.

4. **Displaying the Map**: Finally, we display the map object `m`. In a Jupyter
notebook environment, simply evaluating the map object `m` as the last line of
the cell will render the map directly below the code cell.

Output Explanation:
- The output of the code will be an interactive map displayed directly below
the code cell if you're using a Jupyter notebook. If you're running the code
in a Python script, the map will not be interactive, but you can save it as
an HTML file and open it in a web browser to interact with it.
- The map will be centered at the specified location (London, England) with a
marker placed at the same location. If you click on the marker, a popup with
the text "London" will appear.
- You can zoom in and out of the map using the zoom controls, and you can
click and drag to pan the map. The marker remains fixed at its specified
location as you interact with the map.'''
```

In [ ]:
```
#assignment
#create a map with a marker at Idupulapaya
```

In [7]:
```
#Ans:
import folium

# Create a map centered at a specific location
m = folium.Map(location=[14.276400, 78.536270], zoom_start=10)

# Add a marker at a specific location
folium.Marker(location=[14.276400, 78.536270], popup='idupulapaya').add_to(m)

# Display the map
m
```

Out[7]: Make this Notebook Trusted to load map: File -> Trust Notebook

In [ ]:
```
'''choropleth:
 Choropleth maps provide a visually intuitive way to represent quantitative dat
 By shading or coloring geographic areas based on the values of the variable
 being mapped, choropleth maps allow viewers to quickly grasp
 the spatial patterns and variations in the data.'''
```

In [21]:
```
import folium
import pandas as pd

# Load data (example data for demonstration)
data = pd.DataFrame({
    'State': ['California', 'Texas', 'New York', 'Florida', 'Illinois'],
    'Unemployment Rate (%)': [8.2, 6.9, 9.7, 7.1, 7.3]
})

# Load GeoJSON data for US states (example data for demonstration)
geo_json_data = 'https://raw.githubusercontent.com/python-visualization/folium/

# Create a map centered at a specific location
mymap = folium.Map(location=[37.7749, -122.4194], zoom_start=4)

# Add choropleth layer to the map
folium.Choropleth(
    geo_data=geo_json_data,
    data=data,
    columns=['State', 'Unemployment Rate (%)'],
    key_on='feature.properties.name',
    fill_color='YlOrRd',
    legend_name='Unemployment Rate (%) by State'
).add_to(mymap)

mymap
```

Out[21]: Make this Notebook Trusted to load map: File -> Trust Notebook

In [ ]:
```
'''We create a DataFrame data containing example data.
This DataFrame contains two columns: 'State' and 'Unemployment Rate (%)', repre
We load GeoJSON data representing the boundaries of US states.
We create a map centered at a specific location (San Francisco)
with an initial zoom level of 4.
We add a choropleth layer to the map using folium.Choropleth().
We provide the GeoJSON data, the DataFrame containing the data to visualize,
columns specifying which columns from the DataFrame to use, and various
styling options such as fill_color, fill_opacity, and legend_name.
geojson data is a data format for geographical data
The key_on parameter tells Folium how to match the data being mapped with
the corresponding geographic regions in the GeoJSON data.
In this example, key_on='feature.properties.name' specifies that the data
should be matched with the features in the GeoJSON data based on the "name"
property of each feature. Folium will use this property to associate the
data being mapped with the corresponding geographic regions.'''
```

In [22]:
```
```

Out[22]: —

In [ ]:
```
```